REFERENCE EDITION

Security Operating Manual

A reference system for identity, intelligence, detection, response and automation

Raghav Dinesh

Curated handbook and offline reference workflow

Review edition | 31 October 2026

Contents

A security program you can inspect 4

Audience and scope . . . 4

The Northstar exercise . . . 4

Read, run and inspect . . . 4

How to interpret the recommendations . . . 5

Chapter 1. Security Context and Identity 6

1.1 Start with the decision . . . 6

1.2 Choose access controls deliberately . . . 6

1.3 Operate the identity lifecycle . . . 7

1.4 Session boundaries and protected identities . . . 7

1.5 Telemetry contracts and data quality . . . 8

1.6 Build the capability in stages . . . 8

1.7 Handoff to intelligence and detection . . . 8

Chapter 2. Threat Intelligence and Prioritization 9

2.1 Define the intelligence requirement . . . 9

2.2 Use a time-bounded indicator contract . . . 9

2.3 Confidence is not priority . . . 10

2.4 Handling and dissemination . . . 10

2.5 Turn intelligence into a testable hypothesis . . . 11

2.6 Measure value and maintain the program . . . 11

Chapter 3. Detection Engineering 12

3.1 Specify the rule before tuning it . . . 12

3.2 Normalize without inventing evidence . . . 12

3.3 Choose a data architecture from workload . . . 13

3.4 Test evidence boundaries . . . 13

3.5 Use metrics with correct denominators . . . 14

3.6 Tune and retire responsibly . . . 14

3.7 Coverage and behavioral analytics . . . 14

Chapter 4. Investigation and Response 15

4.1 Prepare authority before an incident . . . 15

4.2 Investigate the Northstar case . . . 15

4.3 Preserve evidence and its limits . . . 16

4.4 Choose containment from scope and consequence . . . 16
4.5 Communicate what the evidence supports . . . 16
4.6 Recovery and learning . . . 17
4.7 Exercise the operating process . . . 17

Chapter 5. Automation and Continuous Improvement 18

5.1 Select work that has a stable decision boundary . . . 18
5.2 Make execution state explicit . . . 18
5.3 Approval and the response boundary . . . 19
5.4 Retries, partial failure and recovery . . . 19
5.5 Treat operational logic as reviewed code . . . 19
5.6 Build an improvement loop . . . 20
5.7 Extend only when the boundary is ready . . . 20

Reference Architecture and Exercise Contract 21

Components and boundaries . . . 22
Data contracts . . . 22
Expected exercise result . . . 23
Production gaps and extension requirements . . . 23
Primary references . . . 23

A security program you can inspect

Security operations become useful when a decision can be traced back to evidence, an accountable owner, and a tested action. This manual connects identity, intelligence, detection, investigation and automation through one small reference system.

Audience and scope

This edition is for security engineers and technical leads who need to connect capabilities that are often deployed separately. It assumes familiarity with authentication, logs and basic Python. It provides operating procedures and design questions, plus an offline implementation that can be reproduced on a laptop. It does not require a commercial SIEM, cloud account, API key or paid model.

The five chapters form a complete, bounded reference edition. The original repository contains longer drafts with vendor comparisons, broad maturity models and planning material. Those drafts remain available for comparison, but they are not part of this publication and contain invalid YAML and unverified historical claims. This edition deliberately replaces universal numeric targets with decisions that readers can test in their own environment.

The Northstar exercise

Northstar is a fictional organization. A finance identity can read a restricted ledger export. An unusual-device authentication from a synthetic intelligence indicator is followed by two successful reads in the same session. The workflow produces a case and simulates revoking that session after a synthetic approval. A normal session, an unrelated session, a public-object read, a duplicate and a malformed record test its boundaries.

What the evidence establishes

The exercise demonstrates a suspicious session and sensitive object access. It does not prove malware execution, credential theft or external exfiltration. Identities, owners, indicators and events are invented for the exercise; none is a customer incident or a production result.

Read, run and inspect

1. Read the architecture

Start with the reference architecture to understand the information and trust boundaries.

2. Run the lab

From the repository root, run the commands below with Python 3.11 or newer. The lab itself uses only the Python standard library.

3. Inspect the result

Read report.json, run-manifest.json and response-state.json. Trace each finding back to its event identifiers.

4. Change a condition

Expire the intelligence, remove object-store events or change the threshold in a copy of the fixture. Observe the changed finding and data-quality output.

```
python -m lab --simulate-approved
python -m unittest discover -s tests -v
```

Run the offline exercise and its tests.

The default output is build/lab/. Running the command again reuses that directory and reports that the response was already simulated. Use a fresh output directory to begin a new exercise. The evaluation clock is the fixture's fixed as_of value, so the example remains reproducible after its synthetic indicator expires in real time.

How to interpret the recommendations

Unattributed procedures and design choices in this manual are recommendations for review, not industry mandates. Primary references are linked where a framework or external standard is discussed. A reader adopting the design must assign owners, validate provider behavior, establish retention and privacy rules, and test its assumptions. The evidence is an executable explanation of a narrow workflow; it is not a claim of production readiness.

Chapter 1. Security Context and Identity

An alert about an account is incomplete until someone can explain whose account it is, what it can reach, and what a response would interrupt. Build that context before automating decisions.

1.1 Start with the decision

The reference decision is whether to investigate and potentially revoke one suspicious session. That decision requires an identity owner, a session identifier, the accessed resource, its sensitivity, and evidence of activity. The lab models these as stable identifiers instead of joining on display names or email strings. Real estates need source identifiers and documented reconciliation because names, aliases and ownership change.

Minimum ownership record

Identity

Stable source identity ID, identity type, accountable owner and lifecycle state.

Access

Approved role or grant, resource scope, purpose, approval reference and expiry where applicable.

Resource

Stable asset ID, business owner, classification, environment and dependencies.

Evidence

Authoritative source, collection time, last successful reconciliation and uncertainty or missing fields.

A service account needs an accountable owner even when no person logs in with it. Include the workload, credential issuer, privilege boundary and rotation or renewal mechanism. An unowned credential should become a tracked exception with an expiry and remediation owner. Assigning a generic team label is insufficient if nobody can approve changes or respond to an incident.

1.2 Choose access controls deliberately

Table 1.1. Access models and their operating trade-offs

Model	Useful when	Review carefully
Role-based access	Job functions have stable permission sets	Role proliferation and inherited permissions
Attribute-based access	Resource and request context affect decisions	Attribute freshness and policy explainability
Relationship-based access	Sharing follows relationships between objects and people	Graph changes, inheritance and revocation behavior

Authentication establishes evidence about a subject. Authorization decides whether that subject may perform a particular action. Federation can reduce separate login systems, but it creates a trust relationship that needs explicit issuer, audience, lifetime and deprovisioning behavior. A successful login does not establish that every subsequent request is appropriate.

NIST SP 800-207 describes zero trust around resources, users and assets, and rejects implicit trust based solely on network location or ownership. Use that framing to ask what each resource request actually verifies. Network reachability, device ownership and a prior login are context; none should silently grant unlimited authority. See Zero Trust Architecture.

1.3 Operate the identity lifecycle

1. Join

Create access from an approved purpose. Record the role, resource scope, owner and start condition. Confirm the authoritative person or workload identity before provisioning.

2. Move

Recalculate required access after a role change. Remove obsolete grants instead of only adding new ones. Ask resource owners to review exceptions that survive the move.

3. Leave

Disable further authentication, address existing sessions and tokens, and reconcile downstream resources. Test each integration; a directory change does not prove every application revoked access.

4. Reconcile

Compare intended access with observed accounts and grants. Queue orphaned identities, failed provisioning and unexpected privilege changes for an accountable owner.

5. Recover

Use a documented emergency-access process with protected credentials, explicit authorization, monitoring and a post-use review. Test recovery without depending on the broken primary identity system.

1.4 Session boundaries and protected identities

A targeted session action has a different blast radius from disabling an entire identity. The lab binds a case to identity_id and session_id, then records a simulated revoke_session action. It does not contact an identity provider. In a real integration, establish what revocation actually invalidates, how quickly applications enforce it, and which refresh tokens or credentials remain usable.

Revocation is not an operation that can simply be undone. Recovery usually means issuing a new session after verifying the subject and removing the cause of compromise. The simulation records that distinction instead of promising a reversible action. Protect emergency and critical workload identities from routine automatic containment and route them to a separate decision process.

Northstar context

Identity

analyst-01 belongs to the Synthetic Finance Team and has the finance-reader role.

Resource

ledger-export is classified restricted. public-guide is classified public and is excluded from the sensitive-read total.

Protected identity

backup-01 models a critical workload identity. The response simulator blocks it even with a matching approval.

Boundary

The fixture context is assumed authoritative. The lab does not discover grants, perform access reviews, enforce authorization or implement an identity provider.

1.5 Telemetry contracts and data quality

Write the log contract with the team that owns the source. Required fields must have defined meanings: actor identity, session, event kind, outcome, resource, source address, occurrence time and receipt time. Preserve the original source event identifier. Document whether bytes means a successful response payload, requested size, compressed transfer or a provider estimate; these are not interchangeable.

The lab requires timezone-aware timestamps and occurred_at <= received_at <= as_of. This is a strict fixture rule, not a universal assumption about distributed clocks. A production pipeline should measure clock skew and implement an explicit tolerance or quarantine policy. Do not silently replace an invalid occurrence time with collection time, because that changes correlation semantics.

- Keep occurrence and receipt times separately so delayed delivery remains visible.

- Track source health and freshness separately from alert volume. A source with one old event can still be unhealthy.

- Treat unknown identities and assets as data-quality failures. Do not silently assign them to a generic owner.

- Separate raw evidence from normalized fields and derived context. Record the transformation version.

1.6 Build the capability in stages

1. Inventory

Start with one sensitive resource and its identities. Resolve ownership and record exceptions before expanding.

2. Reconcile

Compare intended grants with observed access. Test joiner, mover and leaver paths, including failures.

3. Observe

Confirm authentication and resource audit records can be connected through stable identifiers. Measure missing-field and delivery-delay distributions.

4. Exercise

Run a session-compromise tabletop. Identify who can approve containment and how a legitimate user regains access.

Use evidence of capability instead of a single maturity score: can the team identify the owner, explain the grant, retrieve the events, and revoke the relevant access? Report the proportion of in-scope identities with verified ownership, overdue access reviews and failed revocation tests. State the denominator and observation period; avoid arbitrary claims that every program must reach the same percentage.

1.7 Handoff to intelligence and detection

Identity context supplies the business meaning that a network indicator lacks. The next chapter explains how intelligence contributes a time-bounded hypothesis. Threat Intelligence and Prioritization (page 9) starts with the decision that intelligence must support.

Chapter 2. Threat Intelligence and Prioritization

Intelligence is useful when it changes a decision and the team can explain why. A feed entry should carry provenance, validity and uncertainty through every downstream use.

2.1 Define the intelligence requirement

For Northstar, the requirement is narrow: should a successful unusual-device login receive higher investigative priority when its source address appears in relevant intelligence? The indicator is one part of a correlation, not an instruction to block traffic. The follow-on access to restricted data supplies additional evidence. The exercise does not infer a named threat actor from an address.

A useful requirement names the consumer, the decision, the affected assets, the time horizon and the action that might follow. Strategic intelligence can inform resource allocation; operational intelligence can inform a campaign hypothesis; tactical information can support a particular investigation or rule. Collect only what the team can validate and use within that requirement.

1. Direct

Write the question and intended consumer. Define what evidence would change the priority or action.

2. Collect

Select sources that cover the relevant environment. Record original references, observation dates and usage restrictions.

3. Process

Normalize observable types, preserve provenance and deduplicate without destroying distinct observations.

4. Analyze

Separate observed facts from inference. Consider benign alternatives, contradictory reporting and source dependence.

5. Disseminate

Send a decision-focused product with confidence, expiry, restrictions and the recommended next step.

6. Evaluate

Ask the consumer whether it changed the decision. Record irrelevant or harmful matches and revise the requirement.

2.2 Use a time-bounded indicator contract

Indicator record

Identity

A stable indicator ID and typed value such as an IP address or domain.

Provenance

The original source and observation context. Several feeds repeating one report are not independent corroboration.

Validity

An explicit valid_from and valid_until interval. Local policy must define how expiry is assigned and reviewed.

Assessment

Confidence, relevance to the requirement and known benign uses or shared-infrastructure risks.

Handling

Sharing label and any additional restrictions that must survive export and enrichment.

The fixture uses 203.0.113.44 as an invented exercise indicator and labels its source synthetic-exercise-feed. It is not live malicious infrastructure. The lab accepts only high-confidence IP indicators valid both at the authentication event and at the fixed evaluation time. This deliberate conservative policy makes expiry easy to inspect. A historical hunt may need a different policy that preserves and evaluates the intelligence known at the time.

Exact IP comparison avoids the ambiguity of substring matching. Domain and URL indicators need separate normalization rules: hostname case, internationalized names, subdomain boundaries, ports, paths and query components have different semantics. Never lowercase an entire URL and assume identity is preserved. This implementation handles IP indicators only.

2.3 Confidence is not priority

Confidence describes the strength of an assessment. Priority also depends on exposure, asset importance, observed behavior and response cost. High confidence that an address has hosted malicious activity does not prove every user of that address is malicious. Shared hosting, VPNs, reassignment and stale reporting can all create misleading matches.

Table 2.1. Separate observations from conclusions

Observation	What it supports	What remains unknown
Address matches valid intelligence	A reason to enrich and investigate	Whether this session is malicious
Login from an unusual device	A deviation from supplied device context	Whether a legitimate device changed
Restricted reads in the same session	Sensitive access after authentication	Whether data left the approved boundary
Several correlated observations	Higher investigative priority	Root cause and full incident scope

An analyst should be able to inspect each source rather than just a synthetic confidence score. Avoid multiplication of arbitrary feed scores into a pseudo-probability. If a score is used for ranking, document its purpose, inputs and calibration limits. Preserve the underlying facts so a reviewer can disagree with the ranking.

2.4 Handling and dissemination

FIRST TLP version 2.0 uses TLP:CLEAR, TLP:GREEN, TLP:AMBER and TLP:RED; TLP:AMBER+STRICT restricts sharing to the recipient organization. Apply the source's handling instructions and consult the full FIRST TLP guidance before sharing. TLP is a sharing convention, not a substitute for access control or encryption.

The public lab accepts TLP:CLEAR only and reports other labels as intelligence errors. This is a narrow example policy, not a general implementation of TLP permissions. Real sharing requires audience

checks, minimization, redaction and an export path that preserves restrictions. Do not publish raw customer telemetry alongside a sanitized indicator.

Decision-focused intelligence brief

Requirement

Explain which asset or decision the intelligence concerns.

Evidence

List the actual observations, original references and relevant time intervals.

Assessment

State the inference, confidence, competing explanation and what would change the assessment.

Action

Name the consumer, recommended next step, decision deadline and expiry or review date.

2.5 Turn intelligence into a testable hypothesis

The lab hypothesis is that an unusual successful login from a valid high-confidence indicator followed by a large restricted-data read in the same session deserves a case. It is intentionally more specific than matching every indicator against every log. The subsequent Detection Engineering chapter (page 12) specifies the window, threshold and evidence joins.

- Test an expired indicator: it should not contribute to the finding.

- Test missing provenance: reject the indicator visibly.

- Test a low-confidence match: record its policy treatment rather than silently upgrading it.

- Test shared or benign activity: an indicator match alone should not trigger this rule.

- Test the indicator at exact expiry: this lab uses a half-open interval, so valid_until itself is expired.

2.6 Measure value and maintain the program

Measure requirement coverage, time from observation to usable analysis, analyst-confirmed relevance and decisions changed. Track harmful or irrelevant matches alongside useful ones. Count outputs as operational workload, not as proof of effectiveness. A million collected indicators can still leave the highest-priority requirement unanswered.

Do not calculate realized savings by multiplying an average breach cost by every blocked address. The counterfactual is unknown. Report measured handling time, integration costs, review effort and documented decision examples. If estimating avoided loss, state the assumptions and uncertainty separately from measured outcomes.

Start with a small set of requirements and sources, then add capacity when the team can demonstrate operational use. Advanced attribution, machine learning and deception can be separate programs with their own authority and validation. They are outside this example. A maintainable source review should remove redundant feeds, expire obsolete assessments and revisit collection restrictions.

Chapter 3. Detection Engineering

A detection is an executable hypothesis with evidence requirements, explicit failure behavior and a person responsible for keeping it useful. Its most important interface is often the data contract beneath it.

3.1 Specify the rule before tuning it

SOM-DE-001: suspicious session with restricted reads

Hypothesis

A successful unusual-device login from a currently valid, high-confidence exercise indicator followed by sensitive reads in the same session deserves investigation.

Required telemetry

Authentication and object-read audit records with matching identity, session and source IP. Both occurrence and receipt times must be timezone-aware.

Correlation

Successful restricted-object reads at or after authentication and within 900 seconds inclusive. Aggregate only events for that identity, session and source address.

Threshold

At least 100,000,000 bytes of successful restricted reads. This is an exercise parameter, not a recommended enterprise threshold.

Output

A high-priority case with event records, asset and identity context, intelligence provenance, rule version and a reproducible fingerprint.

Limit

The rule detects a narrow conjunction of signals. It will miss compromise without these signals and does not establish external exfiltration.

Write a benign explanation before shipping the rule. A legitimate analyst may use a new device and download a large ledger. An address may be reassigned or shared. The conjunction justifies investigation under the exercise policy; it does not justify declaring a breach. A useful case keeps both the reason for concern and the alternative explanation visible.

3.2 Normalize without inventing evidence

The lab validates records before correlation. Malformed times, missing identifiers, invalid byte counts, unknown identities and unknown assets are quarantined with reasons. Authentication records must name the identity-provider source and reads must name object-store. These source strings are fixture labels, not cryptographic authentication of a producer. A production ingestion boundary would need authenticated collection and controls against forged records.

Exact duplicates are counted once. If two records share an event ID but differ in content, both versions are excluded and reported. Choosing the first arrival would make results depend on ingestion order and could allow corrupted evidence to win. The lab sorts accepted records by occurrence time and event ID so fixture order does not change the case.

The source payload remains attached to the case. Identity, asset, indicator and policy context are included in the case fingerprint, so a changed assessment does not silently reuse an approval bound to

earlier context. A digest helps identify changes; it does not prove source authenticity, trusted collection or legal chain of custody.

3.3 Choose a data architecture from workload

Table 3.1. Architecture choices beyond the local exercise

Pattern	Useful property	Operating cost or risk
Single local batch	Easy to reproduce and inspect	No continuous ingestion, scheduling or concurrency
Central search platform	Shared investigation and correlation	Ingestion cost, retention planning and query contention
Stream plus durable store	Continuous decisions and replayable evidence	Watermarks, late data, idempotency and state recovery
Federated search	Keeps some data within source boundaries	Uneven schemas, availability and query semantics

Choose retention by investigation needs, legal obligations, privacy and storage cost. Keep search access and evidence retention as distinct requirements. Tiering can reduce cost, but an archive that cannot be retrieved within the incident decision window may not satisfy the operational need. Benchmark representative queries and failure conditions rather than relying on vendor list prices or broad event-volume claims.

For continuous correlation, specify how late events reopen or update a case, when a window is final, how replay avoids duplicate actions and how state survives failure. The batch lab does not implement watermarks, streaming state or durable delivery. Its fixed as_of clock is an explicit simplification that makes tests repeatable.

3.4 Test evidence boundaries

1. Positive behavior

Confirm the expected session produces one case with auth-suspect, read-01 and read-02 as evidence. Verify the restricted total is 110,000,000 bytes.

2. Benign controls

Confirm the legitimate session, public-object read and unrelated session do not contribute to that total.

3. Time and volume boundaries

Test exactly the threshold, one byte below it, exactly 900 seconds and one second outside the window. Test both occurrence and receipt constraints.

4. Data failures

Test conflicting duplicates, missing sources, wrong source kinds, unknown identities and malformed records. Inspect the quality report, not only alert count.

5. Intelligence behavior

Test expiry, low confidence, absent provenance and a validity interval starting after authentication.

6. Operational review

Before any real deployment, replay approved historical samples, review privacy and query cost, and run shadow evaluation with analyst feedback.

```
python -m unittest discover -s tests -v
python -m lab --simulate-approved --output build/review
```

Run meaningful boundary tests and inspect a fresh exercise run.

3.5 Use metrics with correct denominators

Table 3.2. Metrics that should not be used interchangeably

Metric	Definition	Interpretation limit
Precision	True positives / (true positives + false positives)	Requires adjudicated alerts and consistent labels
Recall / true positive rate	True positives / (true positives + false negatives)	Unknown missed attacks make production recall hard to estimate
False positive rate	False positives / (false positives + true negatives)	Depends on the evaluation population
Data-quality rejection rate	Rejected input records / all input records	Can rise because the source changed, not because attackers increased
Decision latency	Decision time minus the chosen reference event time	State whether it includes delivery delay and analyst work

The original draft called an alert-quality target a true positive rate. In that context precision is the relevant measure. This edition makes the denominator explicit. Do not claim production precision or recall from this tiny synthetic fixture. The tests establish specified behavior for known cases; they do not sample a real operating environment.

3.6 Tune and retire responsibly

Investigate the cause of noise before increasing a threshold. Duplicated logs, stale identity context, a known batch workload and an incorrect time field need different fixes. A permanent allowlist can conceal future abuse. Give exceptions an owner, reason, scope, expiry and tests that show which behavior is deliberately excluded.

Version each rule with its data requirements, owner, test fixtures, expected output, tuning history and rollback procedure. Deploy a new version in shadow mode when uncertainty is high. Retire a rule when its purpose is superseded or its required telemetry is unavailable, while recording the resulting visibility gap. Silence is not a success metric.

3.7 Coverage and behavioral analytics

Technique mappings are useful indexes into an evaluation plan. They do not show that all implementations of a behavior are detected. Record the exact procedure exercised, source prerequisites, environment and test date. Add an owner and remediation plan for missing telemetry. A colored coverage matrix without those details can imply more assurance than the evidence supports.

Behavioral baselines and learned models need representative observation windows, evaluation labels, drift monitoring and explicit treatment of changing jobs or workloads. A fixed 30- or 90-day baseline is not universally sufficient. This exercise uses a supplied unusual_device boolean; it does not learn behavior, inspect device posture or implement an ML detector.

Hand the case and its limitations to the analyst. Investigation and Response (page 15) describes how to turn that case into a scoped decision without promoting the initial hypothesis to a fact.

Chapter 4. Investigation and Response

A useful response process preserves what is known, makes uncertainty visible and assigns authority for actions that affect the business. The case is a starting point for that process.

4.1 Prepare authority before an incident

Assign an incident commander, technical investigator, business owner and communications lead. Small teams may combine roles, but the responsibilities must remain explicit. Define escalation routes, backups, emergency access and a channel that remains available if the primary identity system fails. Exercise these arrangements before they are needed.

NIST SP 800-61 Revision 3, published in April 2025, supersedes Revision 2 and places incident response within CSF 2.0 cybersecurity risk management. This manual uses that current framing while retaining practical investigation, containment, recovery and learning procedures. See Incident Response Recommendations.

Decision authority

Investigator

Collects evidence, tests hypotheses and recommends scope and action.

Incident commander

Coordinates the incident, records decisions and invokes the agreed escalation process.

Business owner

Explains service impact, dependencies and recovery priorities.

Action approver

Authorizes the specific containment action under the organization's policy.

Communications lead

Maintains consistent internal and external updates with the appropriate specialists.

4.2 Investigate the Northstar case

1. Validate the signal

Confirm the telemetry contract and source health. Inspect the quarantined malformed event separately. Verify the identity and asset context against authoritative systems in a real incident.

2. Establish a timeline

Read the successful authentication at 10:00 UTC, restricted reads at 10:03 and 10:04, and receipt times. Preserve both timestamps and note that the exercise is evaluated at 10:30 UTC.

3. Test the benign explanation

Ask whether a new approved device or planned data export explains the activity. Confirm independently; an attacker may control the suspect session.

4. Expand scope

Inspect related sessions, credential changes, grants, accessed objects and other identities. The fixture contains only a small event set, so it cannot establish absence of wider compromise.

5. Assess business impact

Identify the restricted resource, amount read, affected process and operational consequence of session revocation. Keep suspected exposure separate from confirmed loss.

6. Record the decision

Write the hypothesis, evidence identifiers, unresolved questions, chosen action, approver and review point. Preserve the rationale even if the initial suspicion is later dismissed.

4.3 Preserve evidence and its limits

Store original exports with collection time, source system, collector identity, access controls and integrity digests. Keep a record of subsequent transformations and access. A digest detects a difference from the version hashed; it does not establish that a log was truthful, that the collector was authorized or that a chain of custody is complete. Legal or forensic requirements need a separately designed evidence process.

The lab writes report.json with the event evidence and context, run-manifest.json with input and report digests, and response-state.json with simulated actions. These files are editable local artifacts and are not a tamper-evident evidence store. Preserve copies before experimenting if you want to compare runs.

4.4 Choose containment from scope and consequence

Table 4.1. Containment choices for a suspected identity incident

Option	Possible benefit	Question before use
Revoke one session	Limits a specific active access path	Which tokens and application sessions remain valid?
Disable an identity	Stops broader access through that identity	Which services or emergency processes will fail?
Restrict a resource	Protects a known data boundary	Can the business continue and can investigators still collect evidence?
Observe briefly with safeguards	May clarify scope before disruption	Who accepts the risk and when must the decision be revisited?

The exercise simulates only session revocation. Approval is bound to the case fingerprint and action, has an issue time and expiry, and names a synthetic reviewer. The response function also checks the kill switch and protected-identity flag. These checks demonstrate policy concepts. The reviewer string is not authentication, the approval is not signed and no independent approver service exists.

Restore access through a new trust decision

A revoked session is not restored by undoing a Boolean flag. Verify the user, remove the compromise path and issue a new authorized session. Account recovery, credential replacement and resource recovery may each need separate owners and validation.

4.5 Communicate what the evidence supports

Example internal update

Status

Investigating one synthetic finance session following an unusual-device authentication and restricted object reads.

Known impact

The fixture records 110,000,000 bytes of successful restricted reads. External exfiltration is not established.

Action

Session revocation has been recommended; the lab can record an approved simulation only.

Uncertainty

Root cause, wider access and the legitimacy of the device are unresolved.

Next update

The incident commander sets a review time based on impact and the next evidence collection step.

Set update frequency according to impact and decision needs, not a universal hourly rule. Keep an agreed record of what was communicated, to whom and when. External notifications require jurisdiction-, sector- and contract-specific assessment with the appropriate specialists; this manual does not provide a universal notification clock.

4.6 Recovery and learning

1. Confirm containment

Verify that the selected access path is actually closed. A successful API response may only mean the request was accepted.

2. Remove the cause

Address the credential, session, device, application or permission weakness supported by the investigation. Do not assume a password reset addresses every token or persistence path.

3. Restore deliberately

Bring access and services back with owner approval and observation. Confirm business functionality and define which evidence would trigger renewed containment.

4. Review the incident

Compare the initial hypothesis with the final assessment. Identify collection, decision and communication failures without using the review to assign blame.

5. Track improvements

Give each action an owner, expected evidence, due date and verification procedure. Close it when the change has been tested, not when a ticket is created.

4.7 Exercise the operating process

Use the lab as a tabletop as well as a code demonstration. Pause before approval and ask who owns the session, who can authorize the action and how to reach the user through an independent channel. Inject an unavailable identity provider, a protected workload identity or contradictory intelligence. Record the decisions that the code cannot make.

Measure time to an informed decision, evidence completeness, action verification and closure of improvement items. Distinguish occurrence time, collection time, detection time, analyst start and containment verification. Reporting a single mean can conceal long delays or missing observations; show distributions and unresolved cases when the data permits.

The Automation chapter (page 19) turns this procedure into a controlled execution boundary. The incident commander remains responsible for the decision that precedes any real action.

Chapter 5. Automation and Continuous Improvement

Automation should make a procedure more dependable and explainable. A fast action with stale context or unclear authority can increase the cost of an incident.

5.1 Select work that has a stable decision boundary

Begin with repetitive evidence retrieval, normalization and case preparation. Write the manual procedure first, identify required inputs and define what a successful result looks like. If experienced analysts disagree about the next step, preserve a review point rather than hiding the disagreement inside a script.

Table 5.1. Build automation around explicit decisions

Task	First automation step	Boundary to preserve
Context lookup	Retrieve source and collection time	Missing context must remain visible
Case preparation	Attach evidence and recommended questions	A finding is not an incident verdict
Response recommendation	Produce an action-specific proposal	Authority and business impact need review
Containment	Use a tested, narrowly scoped adapter	Identity, approval, expiry and result verification

Estimate value from observed task duration, frequency, maintenance and review cost. Separate hours released from cash savings. A script that reduces handling time but increases analyst rework may not improve the process. Avoid using a theoretical breach cost as measured automation return.

5.2 Make execution state explicit

The reference pipeline is a local batch: validate inputs, correlate evidence, assemble cases and optionally simulate approved response. It has no scheduler, queue, network adapter or worker pool. A real orchestrator needs durable delivery, bounded retries, timeouts, integration credentials, rate-limit handling and a dead-letter or review route. Each of these changes the failure model.

Execution record

Identity

A stable case and action identifier, target scope and input version.

Authority

A verified approver or authorized policy decision, its lifetime and the exact action approved.

Progress

Requested, accepted, verified, failed or unknown outcome. Distinguish an accepted request from a completed action.

Recovery

What can be retried, what needs a compensating action and what requires a new human decision.

5.3 Approval and the response boundary

The simulator receives a case, an approval, policy, local state and an explicit clock. It blocks a global kill switch or protected identity, then checks the fingerprint, action, reviewer and approval interval. A matching action already present in state is reported as already simulated. Otherwise a new action record is returned without modifying the caller's state object.

The CLI generates a synthetic approval only when --simulate-approved is supplied. It binds the approval to the case and evaluates it at the fixed fixture clock. Its reviewer name documents an exercise role; it does not prove that a person approved anything. No function revokes real access, sends a message or makes a network request.

- Idempotency is scoped to a case fingerprint and action. Changed evidence or context creates a different case and needs a new decision.

- The local JSON ledger is single-process state. It has no locking or atomic multi-file transaction and is not suitable for concurrent workers.

- The simulator demonstrates checks for its own case objects. It does not authenticate an untrusted case payload or defend against a local file editor.

- Real execution requires authenticated approvals, tamper-resistant state, trusted identity and asset context, and provider-specific verification.

5.4 Retries, partial failure and recovery

Classify failures before retrying. A temporary connection error may justify a bounded retry; an invalid target or expired approval needs a new decision. Use backoff and a retry budget for real integrations, and honor provider rate limits. Retry the same operation identifier rather than creating a new side effect on every attempt.

A timeout after a request may leave the outcome unknown. Query the provider or reconcile the state before retrying an irreversible operation. If the provider does not support idempotency, design a deduplication and reconciliation boundary before enabling automatic action. A successful local log write is not verification of a remote effect.

Rollback is appropriate for a reversible configuration deployment, but it is not a universal property of response. Session revocation needs a new authorized session for recovery. Deleting data may have no safe inverse. Document the recovery procedure and its business prerequisites for each action.

5.5 Treat operational logic as reviewed code

1. Propose

Describe the problem, intended behavior, required telemetry and expected operational impact.

2. Test

Use positive, benign, boundary and failure fixtures. Verify expected evidence and state transitions, not just whether a function returns.

3. Review

Have a maintainer inspect the logic, data contract, response scope and documentation. A passing test suite does not replace review.

4. Stage

Use a shadow or recommendation-only deployment before side effects. Observe data quality and analyst workload.

5. Release

Record the deployed version and a recovery plan. Use least-privileged integration credentials and keep secrets out of source.

6. Observe

Monitor failures, unknown outcomes, stale sources and rejected approvals. Reconcile the action ledger with actual provider state.

Infrastructure and policy definitions deserve the same review discipline. Static scanning can identify some configuration problems, but a passing scan does not establish that runtime identity, network paths and business permissions are correct. Verify the deployed state and test the behavior that matters.

5.6 Build an improvement loop

Operating review

Daily or on call

Check ingestion health, quarantines, failed executions, unknown outcomes and decisions waiting for owners.

Periodic

Review noise causes, exception expiry, source relevance, access reconciliation and response exercise outcomes.

After change

Replay affected fixtures and confirm that documentation and action scopes still match the implementation.

After incident

Turn lessons into owned changes with evidence-based closure. Feed new behavior and failure cases back into tests.

Set the cadence according to service criticality and change rate. Avoid treating a fixed calendar as evidence that the control works. Good review records show what was examined, what was found and how follow-up was verified. Keep operational workload metrics separate from security outcomes so a growing automation count does not conceal a growing failure queue.

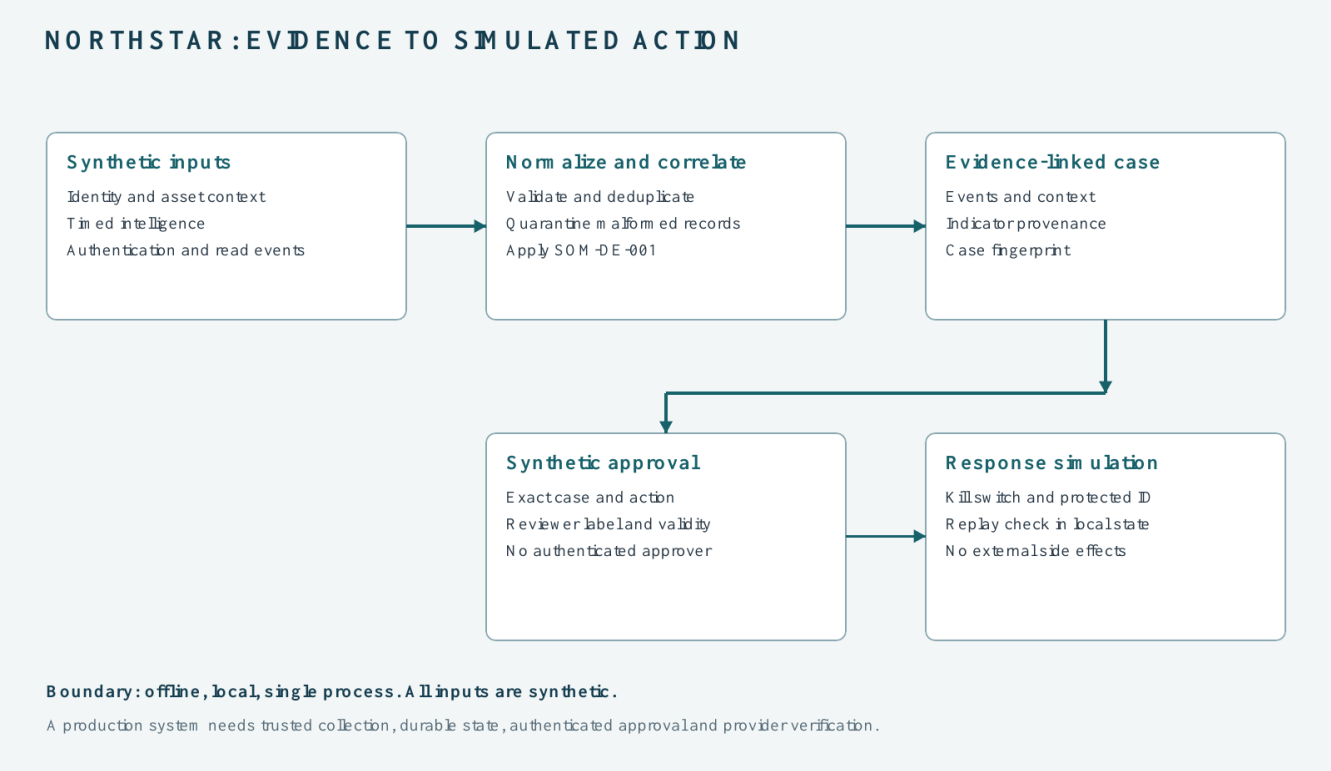
5.7 Extend only when the boundary is ready

The next useful extension is a second scenario with a different benign explanation or missing source, not a dashboard. A real provider adapter would be a separate project milestone requiring an authenticated approval service, credential management, durable transactional state, provider contract tests and a controlled deployment environment.

This edition is complete when the five chapters, architecture, fixture, tests and generated publication agree. Further cloud integrations, machine learning and autonomous action belong to subsequent versions with their own evidence. The current implementation is intentionally sufficient to inspect the operating model from input through decision and simulated action.

Reference Architecture and Exercise Contract

The architecture is a local reference workflow with explicit handoffs. Every component described as implemented is present in the repository; production extensions are identified separately.



Reference workflow and its offline execution boundary.

Components and boundaries

Implemented components

Component	Implemented responsibility	Boundary
Scenario fixture	Synthetic identities, assets, intelligence and events	Assumed source input; no discovery or producer authentication
Normalizer	Schema checks, time checks and event deduplication	In-memory batch; rejected records remain visible
Correlation engine	Same-session restricted-read rule	One rule; no SIEM or behavioral model
Case assembler	Evidence, context, provenance and fingerprint	Editable local evidence; no chain-of-custody service
Approval simulation	Case-bound, time-bounded exercise decision	No authentication, signatures or independent approver
Response simulator	Guard checks and idempotent local action record	No external side effects; single-process ledger

Data contracts

Events

Common

event_id, identity_id, session_id, kind, source, source_ip, occurred_at, received_at and outcome.

Authentication

kind authentication, source identity-provider, outcome success or failure and a Boolean unusual_device.

Object read

kind object_read, source object-store, known asset_id, outcome success or failure and nonnegative integer bytes.

Time

Timezone-aware ISO 8601. The fixture requires occurred_at <= received_at <= as_of.

Duplicates

Identical records with the same event ID count once. Conflicting records sharing an ID are quarantined.

Policy

Rule

rule_id and version are nonempty strings.

Window

window_seconds is a positive integer; the upper correlation boundary is inclusive.

Volume

minimum_bytes is a positive integer. Only successful restricted reads in the matching session count.

Control

kill_switch is Boolean and blocks simulated response when true.

Expected exercise result

The default fixture yields one case for analyst-01 in session-suspect, with 110,000,000 bytes from read-01 and read-02 after auth-suspect. It counts one duplicate and quarantines auth-malformed because its occurrence time lacks a timezone. The public read and unrelated session do not contribute. Missing-source reporting is empty because both required source kinds have accepted records.

The first approved simulation writes one action. A second run using the same output directory records already simulated and leaves the existing action unchanged. An output directory reset removes this local replay memory. This behavior is a fixture-level idempotency demonstration, not a guarantee across independent installations.

Production gaps and extension requirements

- Authenticated collectors and source integrity; immutable raw evidence with retention and access controls.

- Source-specific schema adapters, realistic clock-skew policy and measured delivery delays.

- Durable incremental processing, late-event handling, concurrent transactional action state and reconciliation.

- Authenticated independent approval, business-owner routing and provider-specific containment verification.

- Representative historical evaluation and analyst adjudication before any effectiveness claim.

- Privacy, sharing, retention and notification decisions appropriate to the actual deployment.

Primary references

- NIST SP 800-207: Zero Trust Architecture - conceptual resource and identity trust boundary.

- NIST SP 800-61 Revision 3 - current incident-response risk-management framing.

- FIRST Traffic Light Protocol - sharing labels and usage guidance.

References were checked on 2026-10-31. They support the named framework descriptions, not the lab's arbitrary threshold, performance claims or organizational compliance. No vendor pricing or production effectiveness claim is made in this edition.